

Image Deduplication in Cloud Storage using Cryptographic Techniques

A thesis submitted in partial fulfillment of the requirements for
the award of the degree of

B.Tech.

in

Computer Science and Engineering

By

Adarsh Ashokan (106119001)

Arnav Menon R (106119017)

Ashwath Niranjh A (106119019)



**DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY
TIRUCHIRAPPALLI-620015**

MAY 2023

BONAFIDE CERTIFICATE

This is to certify that the project titled **Image Deduplication in Cloud Storage using Cryptographic Techniques** is a bonafide record of the work done by

Adarsh Ashokan (106119001)

Arnav Menon R (106119017)

Ashwath Niranjh A (106119019)

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **COMPUTER SCIENCE AND ENGINEERING** of the **NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI**, during the year 2022-2023.

Dr. Kunwar Singh

Project Guide

Dr. S. Mary Saira Bhanu

Head of the Department

Project Viva-voce held on _____

Internal Examiner

External Examiner

ABSTRACT

Cloud service providers rely heavily on redundant data storage techniques to ensure data integrity and high availability. However, the storage of duplicate data results in an inefficient use of server space, leading to increased costs and uncertainty during retrieval. In addition, the storage of redundant data poses a security risk, especially when it comes to sensitive information such as images of personal documents.

Image deduplication is a technique that has been proposed to address the problem of redundant data storage in the cloud. It involves identifying and removing duplicate images from the cloud to improve storage efficiency. However, traditional image deduplication techniques suffer from several drawbacks, including the lack of support for images with transformations and the need for high computational resources.

This thesis proposes a new approach to image deduplication that leverages cryptographic methodologies and hashing techniques to compare the image pixel data to identify similarities between images and deny the uploading of duplicates, thus improving storage efficiency.

To ensure secure and transparent record-keeping, the proposed approach utilizes blockchain technology to store all transactions related to copyrighted images. Blockchain provides a tamper-proof and auditable record of all transactions, facilitating dispute resolution.

This thesis proposes a novel approach to address ownership disputes for copyrighted images through arbitration. The approach utilizes image processing techniques to more rigorously compare and verify possible duplicates, and accordingly arbitrates ownership disputes. Additionally, an ad-hoc verifiable proof is generated and published on the blockchain, as a public record to justify successful deduplication. The proposed approach also incentivizes the fair usage of copyrighted images by compensating copyright owners for outsourcing their property, and rejecting unverifiable uploads.

Keywords: Image Deduplication, ZK Proofs, Blockchain, Cloud Storage, Arbitration

ACKNOWLEDGEMENT

We would like to acknowledge the contributions of several individuals and institutions that have made this project possible. Firstly, we would like to express our sincere appreciation to **Dr. Kunwar Singh** for his invaluable guidance and support throughout the project. His insight and expertise have been essential in helping us navigate the various stages of this project.

Our heartfelt gratitude extends to all the members of the internal review panel, **Dr. B. Nithya** , **Dr. A. Santhana Vijayan** , **Dr. Sitara K** whose insight and constructive criticism over multiple extensive reviews shaped the overarching development of this project across all phases.

Our gratitude to the **National Institute of Technology, Tiruchirappalli** and the **Department of Computer Science and Engineering** for giving us the materials and resources to complete this project. We would also like to thank our committee members for their valuable comments and suggestions.

We express our gratitude to everyone who has supported and understood us throughout our research and writing journey. Our family's unwavering support and prayers have been instrumental in helping us reach this point. Last but not least, we are immensely grateful to God for guiding us through the challenges and enabling us to complete our degree. We have felt His presence and guidance every step of the way, and we will continue to trust Him for our future endeavors.

TABLE OF CONTENTS

Title	Page No.
ABSTRACT	ii
ACKNOWLEDGEMENT	iii
TABLE OF CONTENTS	iv
LIST OF FIGURES	vi
LIST OF ABBREVIATIONS	vii
1 Introduction	1
1.1 Aim and Scope of Work	2
2 Review Of Literature	4
3 Preliminaries	7
3.1 Blockchain and Smart Contracts	7
3.2 SPIHT Algorithm	8
3.3 Wavelet Hash (wHash)	10
3.4 RSA Algorithm	11
3.5 Pedersen Commitment	12

3.6	ZK-SNARKs	15
3.7	SSIM Index	16
4	Work Done	19
4.1	Image Deduplication	19
4.1.1	Approach	20
4.2	Incentivization	24
4.3	Arbitration	25
4.3.1	ZK-SNARK Proof Object	28
5	Implementation	30
6	Results and Analysis	33
6.1	Screenshots of Application	33
6.2	Space Saved	35
6.3	Time taken for SPIHT Algorithm	35
7	Conclusion	37
	References	39

List of Figures

3.1	SPIHT Algorithm Overview	9
3.2	Example for wHash	11
4.1	Incentivization Framework	25
5.1	Overall Workflow	31
5.2	SPIHT Workflow	32
5.3	Arbitration Workflow	32
6.1	Application Screenshot 1	33
6.2	Application Screenshot 2	33
6.3	Blockchain Screenshot 1	34
6.4	Blockchain Screenshot 2	34
6.5	SPIHT Matrix Size v/s Execution Time	36

List of Abbreviations

CSP Cloud Service Provider

SPIHT Set Partitioning in Hierarchical Trees

RSA Rivest-Shamir-Adleman

wHash Wavelet Hash

ZK-SNARK Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

SSIM Structural Similarity Index

JPEG Joint Photographic Experts Group

Chapter 1

Introduction

Cloud service providers (CSP) often face the challenge of handling redundant data, which leads to several problems such as wasted resources, poor performance, and increased energy consumption. Redundant data refers to the multiple copies of the same data that exist in a cloud storage system. This can happen due to a variety of reasons, such as data backup, data replication for disaster recovery, and data synchronization. While redundancy is necessary for ensuring data availability and reliability, it can also result in inefficiencies and waste. Additionally, duplicate data requires more energy for storage and retrieval, leading to higher operational costs and carbon footprint.

Traditional image deduplication techniques rely on methods such as checksums, cryptographic hashes, and content-based fingerprints to identify duplicate images. However, these methods are often inefficient when dealing with images that have undergone transformations, such as resizing, cropping, or compression. This is because these techniques are sensitive to even minor changes in the image, which can result in false positives.

Another issue faced by CSPs is the ease with which users can upload images cre-

ated by others and claim them as their own. This is particularly concerning in the case of copyrighted images, where the rightful owner may have no way to dispute the claim of ownership.

The current arbitration techniques for image ownership disputes have not been adapted for cloud-based image deduplication servers. This means that CSPs may have to face legal liabilities if they inadvertently host copyrighted images that have been uploaded by a user claiming ownership.

To address these challenges, CSPs need to incentivize users to utilize their deduplication servers. This can be done by offering discounts or other rewards to users who use the deduplication server to ensure that the original owner's images are not outsourced without their permission. By implementing these measures, CSPs can reduce the amount of redundant data on their servers, improve their performance, and reduce their environmental impact, all while ensuring that copyright laws are respected.

1.1 Aim and Scope of Work

This project aims to develop a cloud-based image deduplication system that utilizes cryptographic techniques such as commitments, zero-knowledge proofs, and digital signatures to provide a secure, fair, and reliable space for storing images. The system also includes incentivization and arbitration schemes to resolve disputes related to ownership and deduplication.

The main objectives of the project are as follows:

1. Development of a module for detecting exact duplicates: Images with the same pixel data can cause redundancy issues for the server. Therefore, an efficient method of comparing images with the existing dataset is necessary to ensure effective deduplication.
2. Development of a module for detecting near-duplicates: With the availability of image editing software, many images are transformed through various techniques such as compression and color transformations. It is crucial to have a systematic approach to deduplicate these images while preserving the transformation data for future retrieval by the uploader.
3. Implementation of incentivization and arbitration schemes: To promote the usage of the platform, incentivizing users to use the system is essential. Additionally, a mechanism for arbitrating disputes related to wrongful deduplication is necessary to ensure fairness and transparency.

Chapter 2

Review Of Literature

In recent years, there has been an explosion in the volume of digital data, making it increasingly challenging to manage and retrieve specific files. Image deduplication is the process of identifying and removing identical or nearly identical copies of an image in a large dataset, which reduces storage costs, improves retrieval speed, and ensures data integrity.

Several approaches have been proposed to address the problem of image deduplication, each with its own strengths and limitations. Some focus on exact duplicates, while others deal with similar images that have undergone some form of transformation. In addition, there are challenges related to ensuring security and privacy, as well as establishing ownership and resolving disputes. In this literature review, we will examine some of the significant works done in the field of image deduplication and highlight their contributions, limitations, and future research directions.

In “Secure deduplication for images using blockchain” by Aparna et al. [2], a framework for image deduplication using blockchain technology has been provided. The paper deals

with exact duplicates and approaches the problem with a rudimentary approach of directly hashing the images and storing it on the database server. Some limitations include

- No security is guaranteed - as the images stored on the cloud server aren't encrypted.

Moreover, the workflow deals only with exact duplicates, and not any transforms of the original image.

Meanwhile, in “Secure image deduplication through image compression,” by Rashid et al. [9], the proposed scheme uses partial encryption with the SPIHT algorithm to ensure security against semi honest CSP for image deduplication, and an image hashing scheme involving the SPIHT algorithm to classify identical compressed and encrypted images for deduplication. However, A full-fledged workflow has not been provided. The paper solely focuses on finding if duplicates exist in the cloud storage using compression, but no proof of ownership or retrieval protocols have been defined.

In order to study how near-transforms can be authenticated in a cryptographic manner, we also went through “PhotoProof: Cryptographic image authentication for any set of permissible transformations,” by Naveh et al. [8]. The proposed approach in the paper for image authentication uses cryptographic proofs that can be configured to allow permissible transformations. The scheme attaches a proof of computational integrity to each legitimate image, attesting that the transformation was permissible, and can be verified by anyone. Further permissible transformations can also be applied, generating updated proofs. One major limitation towards implementing [8] is that the defined protocol is currently limited by existing implementations of ZK-SNARKS. The current runtimes for generation and proving protocols are exponentially proportional to the size of the images.

To further explore how exactly the deduplication server and clients can be incentivised to use the service in a fair manner as well as ensure there aren't any disputes regarding ownership, we studied two other papers - "Blockchain-based deduplication with arbitration and incentives," by Huang et al. [4] and "Official Arbitration with Secure Cloud Storage Application," by Kupcu [7].

[4] proposes to use blockchain technology to create a deduplication system that provides financial incentives for both the server and the user. The proposed system includes an arbitration protocol using chameleon hashes to resolve disputes and ensure fairness. The defined protocols are currently defined for generic files, not images in particular - making it hard for direct implementation.

Meanwhile [7] talks about general arbitration protocols with automated payments using fair signature exchange protocols. Schemes are proposed for static as well as dynamic storage solutions. However, these protocols are designed for privacy within a one client - one server system, and do not account for multiple client situations, such as those in deduplication servers.

Chapter 3

Preliminaries

3.1 Blockchain and Smart Contracts

Blockchain technology is a decentralized digital ledger that allows multiple parties to securely record transactions and information without the need for a central authority. One key feature of blockchain technology is its ability to create "smart contracts," which are self-executing contracts with the terms of the agreement between buyer and seller being directly written into lines of code. Smart contracts can be used to automate a wide range of transactions, from simple payments to complex financial instruments.

Smart contracts work by being programmed with the conditions that must be met for the contract to execute. When these conditions are met, the contract automatically executes, and the agreed-upon outcome is recorded on the blockchain. The contract is transparent, secure, and tamper-proof, and once it is executed, it cannot be changed or reversed.

Smart contracts can be built on various blockchain platforms, including Ethereum, Hyperledger Fabric, and Corda. These platforms allow developers to create and deploy smart contracts that can interact with other smart contracts, as well as with off-chain

data sources and services.

Overall, blockchain technology and smart contracts offer a new paradigm for secure, transparent, and decentralized transactions, with potentially transformative applications in finance, supply chain management, real estate, and many other industries.

3.2 SPIHT Algorithm

The Set Partitioning in Hierarchical Trees (SPIHT) algorithm is a wavelet-based technique that efficiently compresses image data by exploiting redundancy in the image.

The SPIHT algorithm consists of several key stages:

- **Wavelet Transform:** The image is first decomposed into a set of wavelet coefficients using a discrete wavelet transform. This transforms the image into a series of subbands, each containing a different frequency range of the image data. The highest frequency subbands contain the smallest scale features of the image, while the lowest frequency subbands contain the broadest scale features.
- **Sorting:** The wavelet coefficients are then sorted into a list according to their magnitudes. This allows the algorithm to identify which coefficients are the most important in representing the image.
- **Set Partitioning:** The sorted list of coefficients is then partitioned into subsets, or sets. Each set represents a different region of the image. The algorithm determines which sets can be discarded without significantly affecting the image quality, based on a predetermined threshold. The sets that are retained are then

further partitioned, and the process is repeated until the desired compression level is achieved.

- **Encoding:** The sets that are retained are then encoded using binary coding. The encoding process involves coding the locations of the significant coefficients within each set, as well as the magnitudes of those coefficients.
- **Decoding:** To reconstruct the compressed image, the decoding process involves reversing the encoding process. The encoded data is first decoded, and then the wavelet coefficients are reconstructed using the reverse wavelet transform.

This algorithm leverages the statistical properties of wavelet coefficients to exploit redundancy in image data. It is especially effective for images with high spatial redundancy, such as natural images with smooth regions or sharp edges. Furthermore, the SPIHT algorithm has demonstrated superior performance compared to other image compression techniques, including the widely used JPEG standard.

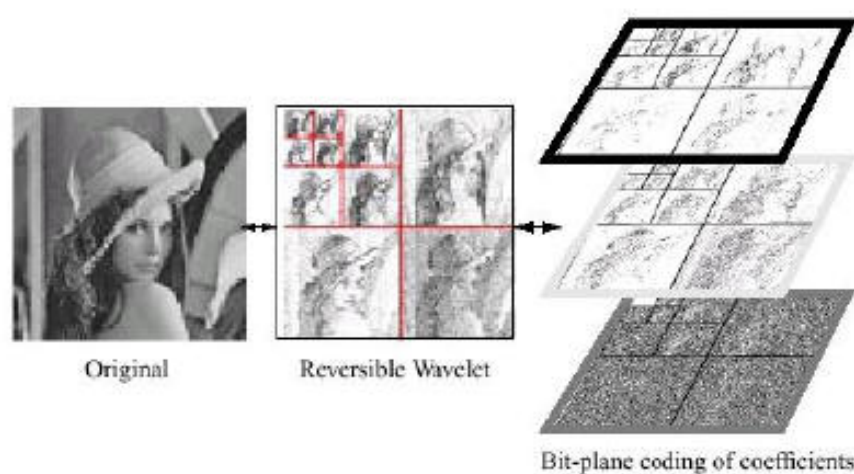


Figure 3.1: SPIHT Algorithm Overview

3.3 Wavelet Hash (wHash)

Wavelet hash is a method used to generate a compact digital signature or hash code for an image that can be used for various applications, such as image retrieval and duplicate detection. The Wavelet Hash algorithm uses Discrete Wavelet Transformation (DWT) to obtain the hash of the image from the wavelet of the image. The scaling effect of DWT provides a great "time frequency representation" where the low frequencies between images appear similar, enabling an accurate comparison of image similarity.

The wavelet transform provides a multi-resolution representation of the image, allowing for more efficient processing of large images. Once the image is decomposed into wavelet coefficients, the algorithm computes the average value of the coefficients in each sub-band. This results in a vector of values that represent the image's frequency content at different scales and orientations.

Next, a threshold value is applied to the wavelet coefficients to create a binary hash code. The threshold value is usually set to the median value of the coefficients, which separates them into two groups: those above the median (which are set to 1) and those below the median (which are set to 0). This results in a binary hash code that captures the essential features of the image's frequency content.

The resulting hash code is highly compact and can be easily compared to other hash codes using bitwise operations. This makes wavelet hash an efficient method for image retrieval and duplicate detection, as similar images will have similar hash codes.

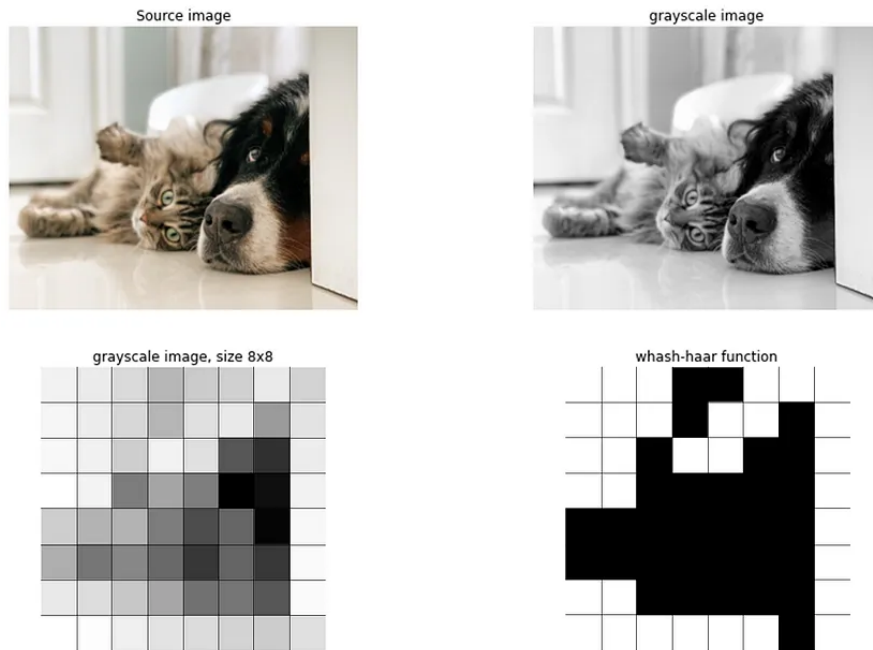


Figure 3.2: Example for wHash

3.4 RSA Algorithm

The RSA (Rivest-Shamir-Adleman) algorithm is a widely-used public-key cryptography algorithm that is used for secure communication and digital signatures. It is based on the mathematical problem of factoring large numbers, which is believed to be computationally difficult.

In RSA, each user generates a pair of keys: a public key, which is shared with others, and a private key, which is kept secret. The public key is used to encrypt messages, while the private key is used to decrypt them.

The encryption process involves taking the plaintext message and raising it to the power of the recipient's public key, modulo a large number known as the "modulus". The

resulting ciphertext can only be decrypted using the recipient's private key.

For example, suppose Alice wants to send a message to Bob using RSA. Bob generates a pair of keys: a public key (e, n) and a private key (d) . The modulus n is the product of two large prime numbers p and q , and e is a number that is relatively prime to $(p - 1)(q - 1)$. Bob shares his public key with Alice.

To send a message, Alice takes the plaintext message M and raises it to the power of Bob's public key e , modulo n , to get

$$C = M^e \bmod n$$

She then sends the ciphertext C to Bob. Bob can decrypt the message by raising the ciphertext to the power of his private key d , modulo n , to get

$$M = C^d \bmod n$$

Overall, RSA is a powerful and widely-used algorithm that enables secure communication and digital signatures in a wide range of applications.

3.5 Pedersen Commitment

The Pedersen Commitment Scheme is a cryptographic construction that allows for one party to commit to a value while keeping it hidden from others until they are ready to reveal it. The scheme was first proposed by Torben Pedersen in 1991 and is based on the discrete logarithm problem in a finite field.

The Pedersen Commitment Scheme is an important tool in modern cryptography, used in various applications such as anonymous credentials, zero-knowledge proofs, and blockchain technology. One of the most notable properties of the scheme is its ability to enable one party to prove that they possess a secret value without revealing the value itself.

To understand how the Pedersen Commitment Scheme works, consider two parties, Alice and Bob. Alice wants to commit to a value without revealing it to Bob, and later she will reveal the value to Bob. She chooses two large prime numbers, p and q , where p is a multiple of $q + 1$, and selects a generator g of the group of order $q, \text{ mod } p$.

To commit to the value x , Alice chooses a random value r in the range $[1, q - 1]$. She then computes the commitment C as

$$C = g^x * h^r \text{ mod } p$$

where h is a publicly known value that is independent of x . Alice publishes the commitment C but keeps x and r secret.

Later, when Alice is ready to reveal the value x , she discloses both x and r . Bob can check that the commitment C is valid by computing

$$C' = g^x * h^r \text{ mod } p$$

and comparing it to the original commitment C . Since the discrete logarithm problem in a finite field is assumed to be hard, it is computationally infeasible to determine x or

r from the commitment C , providing the property of hiding.

The Pedersen Commitment Scheme has several desirable properties. Firstly, it provides hiding, which means that it is difficult to determine the value x or r from the commitment C . Secondly, it provides binding, which means that Alice cannot change the committed value x without revealing r , and vice versa. Finally, it provides additive homomorphism, which means that given two commitments $C1$ and $C2$, it is possible to compute a commitment to the sum of the committed values i.e.,

$$C3 = C1 * C2$$

The Pedersen Commitment Scheme is used in many areas of cryptography, such as anonymous credentials, zero-knowledge proofs, and blockchain technology. In anonymous credentials, the scheme can be used to create digital signatures that allow a user to prove ownership of a credential without revealing any information about the credential itself. In zero-knowledge proofs, the scheme can be used to prove that a statement is true without revealing any additional information about the statement. In blockchain technology, the scheme can be used to enable secure and private transactions.

In conclusion, the Pedersen Commitment Scheme is a powerful cryptographic construction that enables one party to commit to a value while keeping it hidden from others until they are ready to reveal it. The scheme has several desirable properties, including hiding, binding, and additive homomorphism, and is used in various areas of modern cryptography.

3.6 ZK-SNARKs

Zero-Knowledge Succinct Non-Interactive Argument of Knowledge (ZK-SNARKs) is a cryptographic protocol that allows one party to prove to another party that a statement is true without revealing any information about the statement itself. This is accomplished by constructing a proof that can be verified quickly and efficiently, but which does not reveal any information about the statement being proven.

The working of ZK-SNARKs can be illustrated with the following example: Suppose Alice wants to prove to Bob that she knows the solution to a Sudoku puzzle without revealing the solution itself. In a traditional setting, Alice would need to provide Bob with the solution, which would reveal the solution to Bob. But with ZK-SNARKs, Alice can prove to Bob that she knows the solution without actually revealing the solution itself. To do this, Alice first needs to convert the Sudoku puzzle into a mathematical problem that can be represented as a circuit. This circuit is then used to generate a public key and a secret key. The public key is shared with Bob, and the secret key is kept secret by Alice.

Alice converts the Sudoku puzzle into a mathematical problem that can be represented as a circuit. The circuit will have inputs for each cell in the puzzle and outputs that indicate whether the solution is correct or not. This proof is generated using a mathematical technique called a "quadratic arithmetic program" (QAP), which maps the problem to a system of polynomial equations.

Next, Alice uses the secret key to create a proof that she knows the solution to the Sudoku

puzzle using a technique called a "structured reference string" (SRS) to generate a set of "randomized polynomials" that are used to generate the proof. The SRS is essentially a set of precomputed values that can be used to generate proofs for any statement in the language being proven. This proof is a string of data that is created by running a set of computations using the circuit and the secret key. The proof is then sent to Bob.

Bob can then verify the proof by running a set of computations using the public key and the proof. If the computations check out, Bob can be confident that Alice knows the solution to the Sudoku puzzle without actually knowing the solution herself. Bob checks the proof by using a "pairing function" that checks the relationship between two points on an elliptic curve. This function allows the verifier to efficiently verify the proof without having to solve the polynomial equations themselves.

Overall, ZK-SNARKs use a combination of mathematical techniques to generate secure and efficient proofs of knowledge without revealing any information about the underlying statement being proven. This has important applications in areas such as blockchain technology, where privacy and security are essential.

3.7 SSIM Index

The Structural Similarity Index (SSIM) is a widely used image quality assessment metric that is used to measure the similarity between two images. It works by comparing the structural information of an image with a reference image, taking into account luminance, contrast, and structural information. The SSIM index produces a score between

0 and 1, with a score of 1 indicating a perfect match between the two images. Unlike other image quality metrics that rely solely on pixel-level comparisons, SSIM takes into account the perceptual aspects of image quality, making it a more accurate measure of image similarity. SSIM is used in many applications, including image and video compression, digital watermarking, and image restoration.

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

The SSIM formula is based on three comparison measurements between the samples of x and y : luminance (l), contrast (c) and structure (s). The individual comparison functions are:

$$\begin{aligned} l(x, y) &= \frac{2\mu_x\mu_y + c_1}{\mu_x^2 + \mu_y^2 + c_1} \\ c(x, y) &= \frac{2\sigma_x\sigma_y + c_2}{\sigma_x^2 + \sigma_y^2 + c_2} \\ s(x, y) &= \frac{\sigma_{xy} + c_3}{\sigma_x\sigma_y + c_3} \end{aligned}$$

with, in addition to above definitions:

- $c_3 = c_2/2$

SSIM is then a weighted combination of those comparative measures:

$$\text{SSIM}(x, y) = l(x, y)^\alpha \cdot c(x, y)^\beta \cdot s(x, y)^\gamma$$

Setting the weights α, β, γ to 1, the formula can be reduced to the form shown above.

where,

- μ_x : the pixel sample mean of x ;
- μ_y : the pixel sample mean of y ;
- σ_x^2 : the variance of x ;
- σ_y^2 : the variance of y ;
- σ_{xy} : the covariance of x and y;
- $c_1 = (k_1 L)^2$, $c_2 = (k_2 L)^2$: two variables to stabilize the division with weak denominator;
- L : the dynamic range of the pixel-values (typically this is 2# bits per pixel – 1)
- $k_1 = 0.01$ and $k_2 = 0.03$; by default.

Chapter 4

Work Done

4.1 Image Deduplication

Exact and near-duplicate images can cause redundancy in data storage, leading to delays in important user metrics such as retrieval time, along with environmental drawbacks such as excessive energy usage, and emitting high levels of heat. The presence of image editing software, providing a wide range of transformations including compression, cropping, color adjustments, redacting, and more, has further increased the number of transformed images. As a CSP, efficient resource management is a top priority to optimize service. Thus, it is crucial to identify and remove duplicate images to ensure optimal service provision. In addition, it is essential to identify and verify these transformations, if any, to deduplicate images and reduce server space usage by storing only the transformations rather than the entire image. This approach not only saves space but also enables easy retrieval of transformed images by applying the transformation based on the log of the original image.

4.1.1 Approach

An efficient approach for deduplication in both the aforementioned situations can be established using a unified method. Our approach is based on three key assumptions:

- Every image has a digital signature that can be used to verify the ownership information of the image.
- An image transformation software that creates a transformation log when exporting the final image, and has a mechanism to verify the log's correctness, is available
- The EXIF data of the image cannot be tampered with.

The first step in evaluating any image is to ensure that the ownership data of the image is correct and intact. According to our assumptions, the image contains an RSA digital signature that verifies the copyright data of the image. Considering the public key of the user is available publicly,

Client Side

- 1: $X \leftarrow \text{copyright}$
- 2: $\text{sign} \leftarrow \text{RSA}(X, \text{private_key})$

Server Side

- 1: **if** $\text{copyright} == \text{verification}(\text{sign}, \text{public_key})$ **then**
- 2: output right sign
- 3: **else**
- 4: output wrong sign
- 5: **end if**

After the signature verification process confirms the correctness of the signature, the image's pixel data needs to be compared with the existing images on the server. However, comparing the original pixel data of various images can be time-consuming and inefficient, given the varying shapes and sizes of images.

To address this, we use the SPIHT algorithm to compress the images to a set dimension with significantly reduced size. The original image matrix is processed through the algorithm to obtain the haar wavelet of the compressed image, which is then compared with the wavelets of the images already on the server. Once the compressed image wavelet is obtained, a locality-sensitive hash algorithm is utilized to compare similarities between images. In our case, we found that the Wavelet Hash algorithm performed the best in providing an accurate comparison by returning the hamming distance between the hashes of the compared images.

Once the hamming distance is received, based off the threshold value set, two conditions arise:

1. If the images are dissimilar, the image is considered unique and uploaded to the server, along with a transaction added to the blockchain regarding the same, holding the information regarding the fee transfer from user to server.
2. If the images are similar, the transformation log is checked. If no log is present, the image is considered tampered in some way and is rejected, which the user can dispute via arbitration.

If the images are similar, the transformation log is checked. If no log is present, the image is considered tampered in some way and is rejected, which the user can dispute via arbitration.

The user submits a transformation document which contains the Pedersen commitment of the transformation log. The transformation log can best be explained with the following example:

1	c		1429
2	M		GB 1; S; B
3	r		7
4	g		5
5	q		4667
6	h		4454

where,

- c - The Pedersen Commitment of the transformation log
- M - The transformation log denoting the list of operations along with required parameters (eg. GB 1 denotes Gaussian Blur with a radius of 1)
- The public values required to verify the commitment:
 - q - q is of form $2p + 1$, where p is any large prime number
 - g - the generator of the cyclic group formed by p
 - $h - g^s \bmod q$

In our case, the public values q , g , h are made available to the server through the transformation document. We parse the document and retrieve the required values before

continuing to the next step of the verification.

If the verification returns positive, we store the log, and add a transaction on the blockchain with the log of transformations and send a fee from the uploader to the server and the owner of the image. And in case it's wrong, we reject the image.

```
1:  $x \leftarrow image\_comp\_val$ 
2: if  $x < threshold$  then
3:    $upload\_img()$ 
4:    $upload\_txn()$ 
5: else
6:   if  $transformation\_log\ exists$  then
7:     if  $verify\_commitment\ is\ True$  then
8:        $upload\_log()$ 
9:        $upload\_txn\_for\_log$ 
10:    else
11:       $reject$ 
12:    end if
13:  else
14:     $reject$ 
15:  end if
16: end if
```

The helper functions in the algorithm above are as follows:

- **upload_img()** - this function takes the image that the user had uploaded and

uploads the image into the cloud server.

- **verify_commitment()** - this function takes the public values, the transform log and the commitment as input and performs the verification algorithm of Pederson Commitment and returns whether the verification has turned out correct or not.
- **upload_txn()** - this function helps transfer the fee for uploading from the user to blockchain, along with adding a transaction containing info of the new image to the blockchain.
- **upload_log()** - this function takes the transformation log of the image which has been identified as transformed image, and uploads the image's log to the cloud server.
- **upload_txn_for_log()** - this function helps transfer fees from the user to the server and the owner of the original image, and adds a transaction with info of the transformation to the blockchain.

4.2 Incentivization

Incentivization models are an essential aspect of cloud storage services, as they motivate users to take certain actions that benefit both themselves and the service provider. There are various types of incentives that can be used to encourage users to use cloud storage services more effectively, including monetary rewards, discounts, premium services, and storage space upgrades.

On uploading a fresh image, the data owner pays a certain fee to the CSP, scaled to the size of the image stored. Further upload of the same image will be linked to the original

image, provided they are correctly attributed, and the Data Renter pays a nominal rent fee to the data owner, and an access fee to the CSP. In cases of possible malpractice, such as incorrect copyright, missing transformation log etc. the upload request is rejected.

The incentives are distributed in the form of ether (ETH) on an Ethereum blockchain. For each successful upload request, a transaction is processed via a smart contract deployed on the blockchain, and the associated ether amount is distributed among the involved parties.

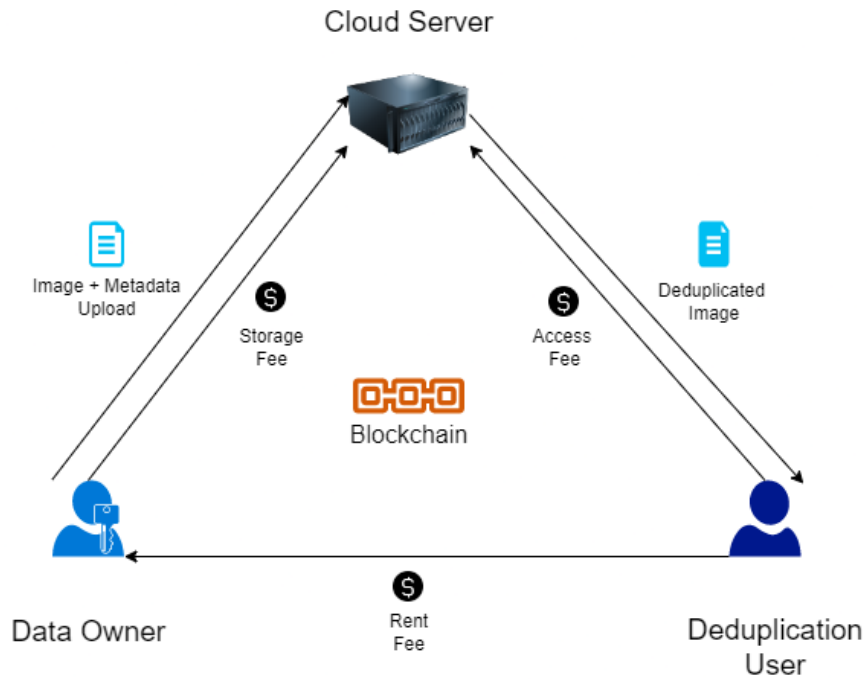


Figure 4.1: Incentivization Framework

4.3 Arbitration

A cloud image deduplication system is an effective tool for managing large amounts of image data stored in the cloud. However, when multiple users or parties upload the same image, ownership and control disputes can arise. Arbitration is a dispute resolution

process that can help resolve such issues.

Arbitration involves a neutral third party facilitating negotiations between parties to reach a mutually acceptable solution. In a cloud image deduplication system, arbitration can be used to resolve ownership disputes when multiple parties upload the same image to the cloud. Ideally, a neutral third party would moderate the arbitration process. However, this may not always be feasible. In such cases, an intermediary arbitration scheme can be used to ensure a fair and transparent process.

The intermediary arbitration scheme involves using publicly verifiable proofs such as a digital signature or a hash function to authenticate ownership claims. Parties can submit their ownership claims with proof of ownership to the arbitration system, which will evaluate the claims and determine the rightful owner of the image. The cloud image deduplication system can then resolve ownership disputes fairly and efficiently without the need for costly legal proceedings, promoting trust and confidence in the system and increasing its adoption and usage.

The user who wants to arbitrate against a refusal by the CSP to upload their image can make use of the Arbitrate feature as follows:

1. In order to ensure that the CSP is not spammed with arbitration requests, the user who wants to arbitrate is required to stake a small amount. This also serves as a means to cover the computing costs of arbitration over the longer run.
2. The uploaded image is compared against the images in the cloud and the closest

image present among the images in the cloud is procured, depending on the set threshold. For ease of understanding, call the uploaded image as *new_img*, and the closest image as *candidate_image*.

3. Once we obtain the *candidate_image*, we then compare the similarities of the two images using the SSIM index, which results in a stricter similarity check than the hash comparison used during the normal upload process.

(a) If the value of the SSIM index is more than or equal to a set threshold

This results in a failed arbitration for the user.

Since it is a failed arbitration, the amount that has been staked is not returned back to the user.

The CSP provides a ZK-SNARK proof object that proves to the user that there exists an image in the database close enough to the uploaded image already in the cloud. This proof object can be used by the client to verify the result of the arbitration.

(b) If it is less than the set threshold

This results in a successful arbitration for the user.

As a result, the staked amount will be returned back to the user. The image will also be uploaded to the cloud, and be attributed to the user who arbitrated successfully against the CSP.

4. The results of the arbitration are then, i.e. the transactions involved are then published to the blockchain.

4.3.1 ZK-SNARK Proof Object

The ZK-SNARK proof object is created by the cloud server in order to prove to the user that there exists an image in the cloud storage that is close enough to the currently uploaded image.

Currently the ZK-SNARK proof works on the simple ZK-SNARK proof of showing one party knows a valid pre-image x , for a hash function value $H(x)$. Over here the public value, is the SHA-256 Hash of the wHash of *new_img*.

The server generates a proof saying that the wHash of the *candidate_image* is a valid pre-image that can give the same image as the value of the *new_img*.

This ZK-SNARK proof is generated only after the SSIM check is run between the *candidate_image* and *new_img* and satisfies the threshold conditions. Only when the SSIM check is valid, do we know that there exists an image in the cloud storage whose whash is as same as the *new_img*.

The ZK-SNARK proof object essentially proves to the user who uploads *new_img*, that the server has an image in the cloud (*candidate_image*) whose wHash is as same as the uploaded images wHash.

The code snippet for the ZK-SNARK proof - coded on Zokrates is as follows as .zok file is as follows:

```
1 import "hashes/sha256/512bitPacked" as sha256packed;
2
3 def main(private field a, private field b, private field c, private
4   field d, field hashDigest1, field hashDigest2) {
5   field[2] h = sha256packed([a, b, c, d]);
6   (h[0] == hashDigest1);
7   (h[1] == hashDigest2);
8   return 1;
9 }
```

Chapter 5

Implementation

The project was implemented through the use of a WebApp that serves as a gateway for users to upload the images to the cloud. The WebApp is built on a Python flask backend, and the frontend was created using HTML, JS and Bootstrap. The entire application was built on Python. The smart contracts utilized were built with Solidity and hosted on a private blockchain simulated by Ganache. The ZK-SNARK was generated using Zokrates, a toolbox for deploying ZK-SNARKs on Ethereum, built on the Rust programming language.

The general workflow of the project can be broken down into 3 parts:

1. **Signature verification and hash comparison:** This part consists of verifying the digital signature in the image for ownership, and going forward with comparing the hashes obtained after SPIHT algorithm and wavelet hashing methods. This helps determine whether to go forward with the workflow based on the scenario.
2. **Commitment verification and transformation analysis:** This part consists of analyzing the transformation log and verifying the commitment attached for correctness, which is then followed by transacting on the blockchain for recording,

along with incentivizing the concerned parties.

3. **Arbitration:** This part allows the user to dispute the deduplication performed by the server which then is analyzed through a more rigorous procedure to retrieve a more accurate evaluation of the deduplication. Based on the success of the arbitration, either the user's image is uploaded and the stake invested is returned, or the stake is passed onto the server and the image rejected.

The general workflow diagram of the project is as follows:

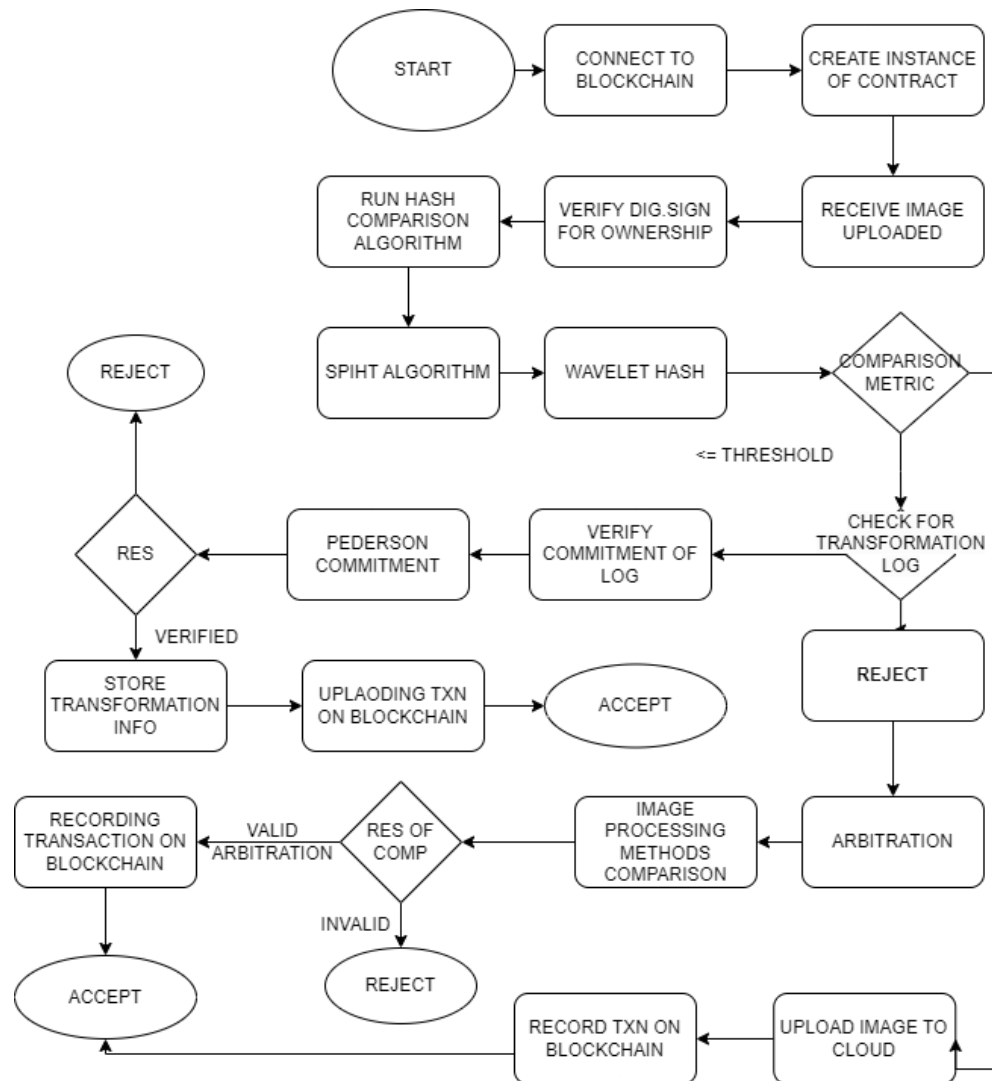


Figure 5.1: Overall Workflow

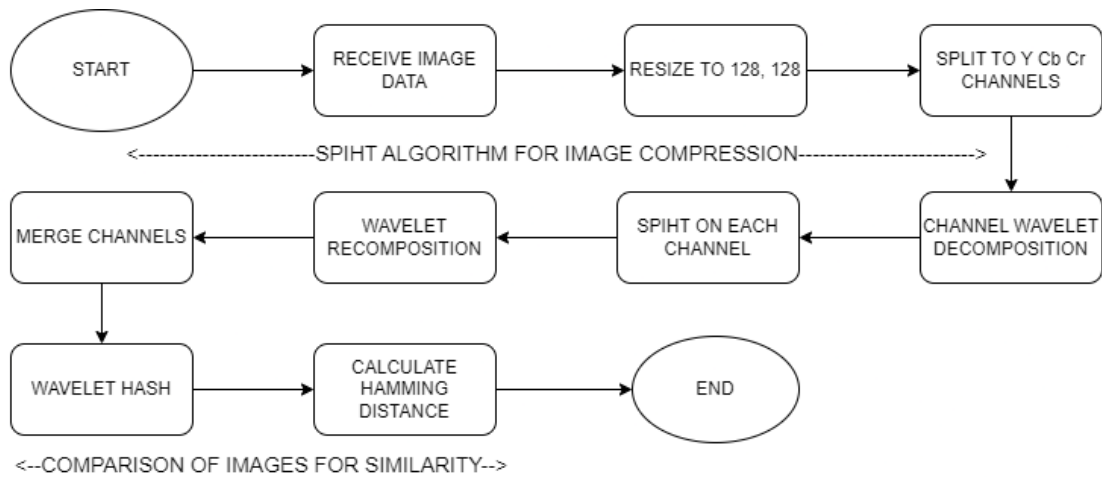


Figure 5.2: SPIHT Workflow

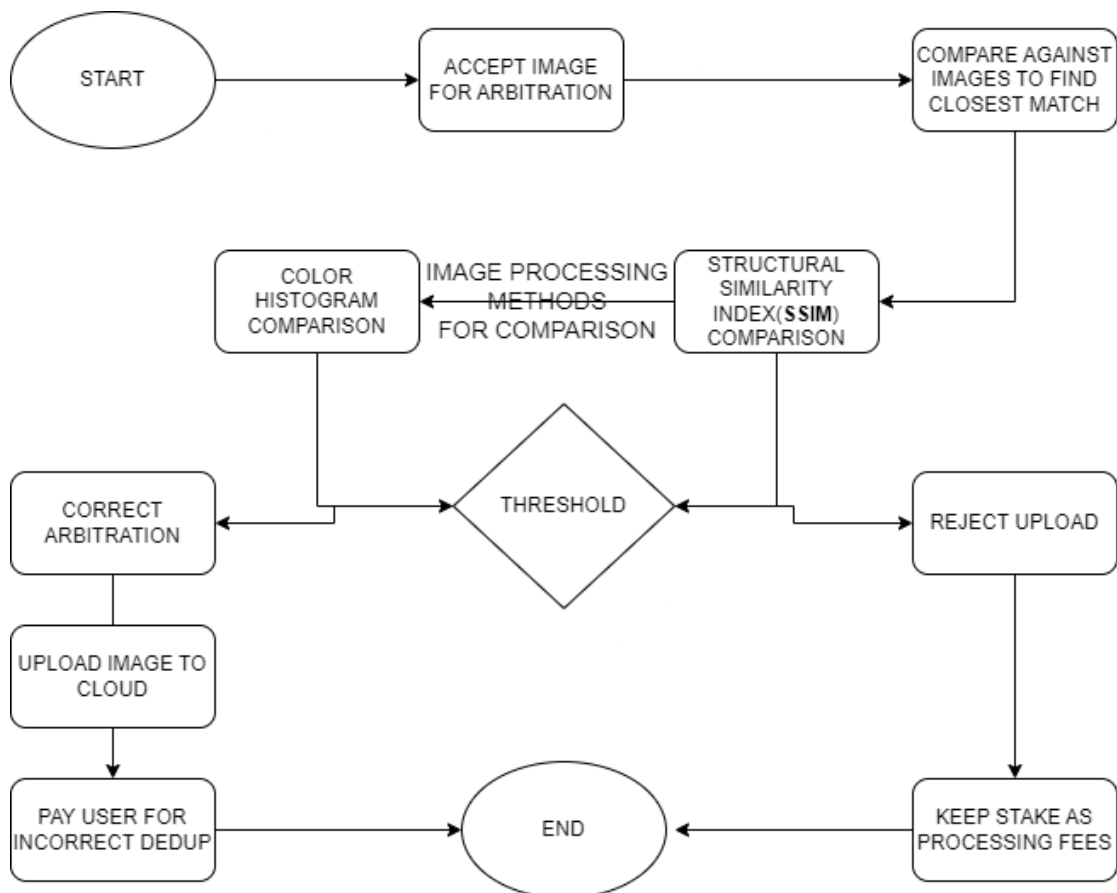


Figure 5.3: Arbitration Workflow

Chapter 6

Results and Analysis

6.1 Screenshots of Application

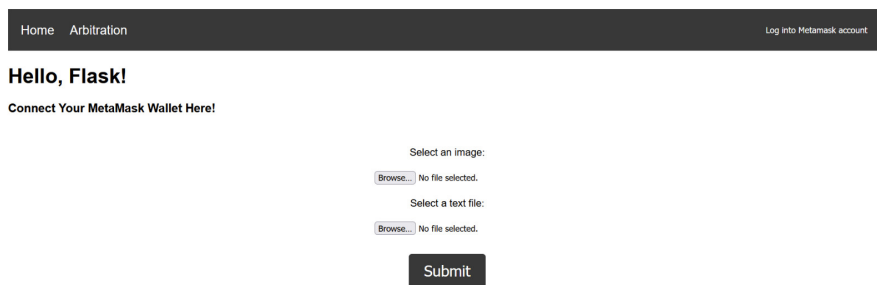


Figure 6.1: Application Screenshot 1

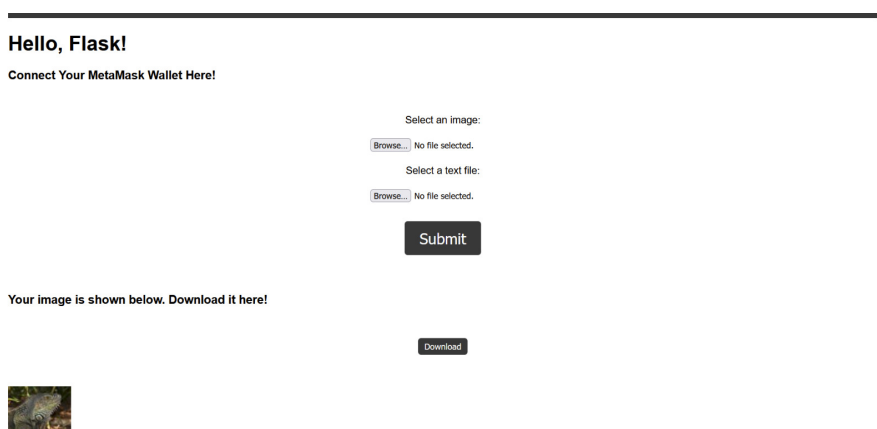


Figure 6.2: Application Screenshot 2

6.2 Space Saved

The PoC was simulated with a database of 150 images with custom EXIF tags, and various transforms. Each image was manually added to the cloud server wherein only original images were stored in the dataStorage folder.

Following this, we observed that the deduplication server **saved 34.77% space on average** over the 150 images, as it only stored the original images and not any transform.

6.3 Time taken for SPIHT Algorithm

Since the image compression SPIHT algorithm is the most compute-intensive part of the workflow, we measured the average time it would take to compress an $N \times N$ image where the $N \times N$ refers to the dimension of the pixel matrix of the image.

The values for the graphs were calculated as follows. Each image in the dataset was resized to 2x2, 4x4, 8x8, 16x16 and so till 512x512. Following which SPIHT algorithm was run on them. For each size $N \times N$, the average time for SPIHT is the average of the time taken by each image when resized to $N \times N$.

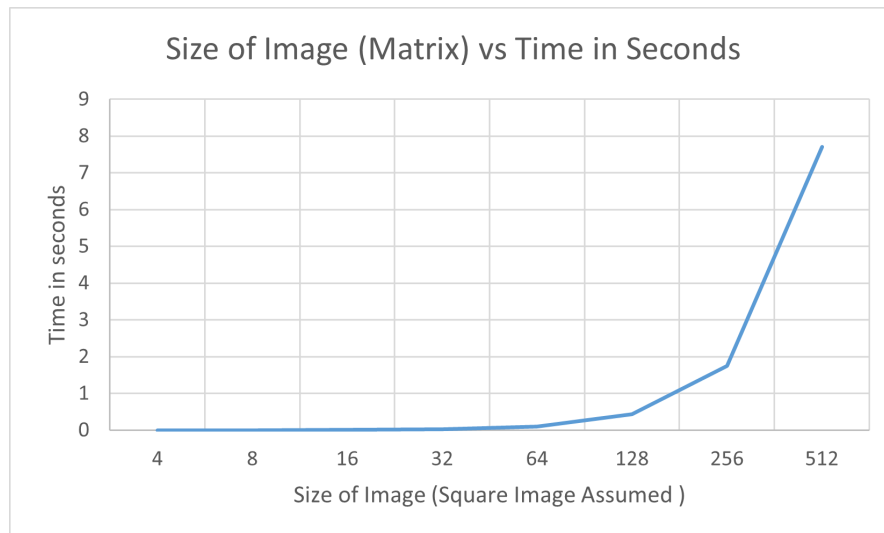


Figure 6.5: SPIHT Matrix Size v/s Execution Time

Chapter 7

Conclusion

We have successfully implemented a cloud-based image deduplication system that employs cryptographic techniques and hashing algorithms to identify and remove duplicate images from the cloud, utilizing blockchain technology to ensure secure and transparent record-keeping of all transactions related to copyrighted images, facilitating dispute resolution. The system includes incentivization and arbitration schemes to promote the usage of the platform and ensure fairness and transparency.

Our approach employs a robust methodology that encompasses several crucial steps to ensure the authenticity and uniqueness of images. To verify ownership data, the RSA digital signature scheme is utilized. The images are compressed using the SPIHT algorithm, and their wHashes are compared with other images already present on the server to prevent duplication. A key feature is the storage of only the log of transformations, rather than the entire image, resulting in efficient space allocation. The correctness of the arbitration protocol is demonstrated through the use of ZK-SNARKs, to publish a succinct and zero-knowledge proof that can be verified at any time.

Future work includes implementing deduplication on encrypted images to improve the security of the cloud service. Expanding the scope of the current representation of the transformation log to cover more complex image operations and incorporating multiple formats of such logs can increase the system's versatility and adaptability. Employing a dynamic fee structure for the various transactional operations can help promote fairness and discourage malicious activity on behalf of all parties involved.

Bibliography

- [1] Daniel Kang, Tatsunori B. Hashimoto, Ioan Cristian Stoica, and Yi Sun. Zk-img: Attested images via zero-knowledge proofs to fight disinformation. *ArXiv*, abs/2211.04775, 2022.
- [2] R. Aparna, Roopa G. Kulkarni, and Shilpa Chaudhari. Secure deduplication for images using blockchain. *2020 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, 2020.
- [3] Mihir Bellare, Sriram Keelveedhi, and Thomas Ristenpart. Message-locked encryption and secure deduplication. *Advances in Cryptology – EUROCRYPT 2013*, page 296–312, 2013.
- [4] Ke Huang, Xiaosong Zhang, Yi Mu, Fatemeh Rezaeibagha, Xiaoming Huang, and Yongcheng Gong. Blockchain-based deduplication with arbitration and incentives. *IET Information Security*, 16(6):401–416, 2022.
- [5] Hankyung Ko, Ingeun Lee, Seunghwa Lee, Jihye Kim, and Hyunok Oh. Efficient verifiable image redacting based on zk-snarks. *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*, 2021.

- [6] Lu Chen, Feng Xiang, and Zhixin Sun and. Image deduplication based on hashing and clustering in cloud storage. *KSII Transactions on Internet and Information Systems*, 15(4):1448–1463, April 2021.
- [7] A. Kupcu. Official arbitration with secure cloud storage application. *The Computer Journal*, 58(4):831–852, 2013.
- [8] Assa Naveh and Eran Tromer. Photoproof: Cryptographic image authentication for any set of permissible transformations. *2016 IEEE Symposium on Security and Privacy (SP)*, 2016.
- [9] Fatema Rashid, Ali Miri, and Isaac Woungang. Secure image deduplication through image compression. *Journal of Information Security and Applications*, 27-28:54–64, 2016.
- [10] A. Said and W.A. Pearlman. A new, fast, and efficient image codec based on set partitioning in hierarchical trees. *IEEE Transactions on Circuits and Systems for Video Technology*, 6(3):243–250, 1996.
- [11] Sriram Keelveedhi, Mihir Bellare, and Thomas Ristenpart. DupLESS: Server-Aided encryption for deduplicated storage. In *22nd USENIX Security Symposium (USENIX Security 13)*, pages 179–194, Washington, D.C., August 2013. USENIX Association.
- [12] Haoran Yuan, Xiaofeng Chen, Jianfeng Wang, Jiaming Yuan, Hongyang Yan, and Willy Susilo. Blockchain-based public auditing and secure deduplication with fair arbitration. *Information Sciences*, 541:409–425, 2020.